VOTA: Verdict-Optimized Trace Abstraction for Execution-Grounded Test Outcome Prediction

Anonymous

May 15, 2026

Abstract

Trace-augmented supervised fine-tuning (SFT) has shown promise for code understanding, yet prior work evaluates exclusively on generative tasks (code completion, repair) with modest gains. We investigate trace SFT for *verdict tasks*—binary pass/fail prediction of test outcomes—and find a striking task–representation mismatch: full execution traces *collapse* model accuracy to 28.5% (below majority baseline), while a compact, decision-relevant abstraction we call **Verdict-Optimized Trace Abstraction (VOTA)** achieves 99.2% (+24.0pp over no-trace). Through systematic ablation, we show that (1) VOTA teaches execution-grounded verdict reasoning beyond string comparison—removing actual/expected values still yields 89.1% (+14.0pp); (2) full-trace collapse is caused by gradient dilution, confirmed by label-only loss repair and a monotonic accuracy–length decline ($r{=}-0.97$); and (3) all three VOTA components (assertion snapshots, exception summaries, loop compression) contribute independently with super-additive interaction. Results replicate on DeepSeek-Coder-6.7B (+20.3pp) and a second verdict task (DebugBench bug detection, +17.7pp), with seed variance below 0.2pp. Our findings establish that trace information granularity, not presence, determines SFT value for classification, and provide quantitative design principles for task-optimized trace representations.

1 Introduction

Execution traces—step-by-step records of program runtime behavior—encode rich information about code semantics. Recent work on trace-augmented supervised fine-tuning (trace SFT) has explored whether exposing language models to execution traces during training improves code understanding. DCSH [1] tested five trace formats across generative tasks and found limited benefit (+2–5pp). TracePile [2] scaled to 2.6M trace-augmented samples but again evaluated only generation tasks. TRACED [3] demonstrated that traces help encoder-based classification, but decoder-based trace SFT for classification remains unexplored.

A critical gap in this literature is the absence of *verdict tasks*: binary pass/fail classification of code behavior. Verdict tasks have fundamentally different output entropy ($O(1)$ bits) compared to generation tasks ($O(n)$ bits), yet all existing trace formats are designed for full execution simulation. We hypothesize that this mismatch matters: trace granularity must match task output complexity.

We introduce **Verdict-Optimized Trace Abstraction (VOTA)**, a compact trace representation that captures only decision-relevant execution states: assertion-point value snapshots, exception summaries, and loop compression patterns. On test verdict prediction with CodeLlama-7B, VOTA-SFT achieves 99.16% accuracy (+24.03pp over no-trace), while full execution trace SFT *collapses* to 28.49%—a 46.64pp *decrease* that produces degenerate all-fail predictions.

Our contributions are:

1. **VOTA teaches execution-grounded reasoning, not string comparison.** Removing actual/expected values from VOTA traces still yields 89.1% accuracy (+14.0pp over no-trace, $p < 0.001$), demonstrating that assertion structure, exception patterns, and loop behavior carry sufficient verdict-relevant signal.

2. **Full-trace collapse is gradient dilution, not data corruption.** Label-only loss repairs collapse (28.5%→76.2%), and accuracy declines monotonically with trace length ($r = -0.97$). Trace content is not harmful; the loss distribution mismatch is.

3. **All VOTA components contribute independently with super-additive interaction.** Component ablation quantifies each element's contribution (loop: +4.0pp, exception: +7.6pp, actual/expected: +16.1pp) and reveals a super-additivity ratio of 1.15×, providing design principles for task-optimized traces.

Results generalize across models (DeepSeek-Coder-6.7B: +20.3pp), tasks (DebugBench bug detection: +17.7pp), and seeds (std < 0.2pp).

2 Related Work

Trace-Augmented SFT for Code. DCSH [1] systematically evaluated five execution trace formats (variable snapshots, assertion states, full traces, compressed traces, and natural language summaries) for code generation tasks via decoder SFT, finding modest improvements of +2–5pp. TracePile [2] constructed a large-scale dataset of 2.6M trace-augmented code samples and demonstrated that continued pre-training on traces improves code generation, though gains plateau with scale. Both works focus exclusively on generative tasks and do not consider classification-specific trace design.

Traces for Code Classification. TRACED [3] showed that execution traces improve encoder-based models (CodeBERT, GraphCodeBERT) on bug detection and vulnerability classification. However, encoder-based approaches process traces as input features rather than through SFT, leaving the decoder SFT paradigm unexplored for classification tasks.

Test Outcome Prediction. Prior work on test outcome prediction relies on static features (code changes, test history, flakiness signals) without execution trace information [4]. Our work is the first to apply trace SFT to verdict tasks and to design trace representations specifically for classification.

3 Method

3.1 Task Definition

Given source code c and a test case t, the *test verdict prediction* task requires predicting the binary outcome $y \in \{\texttt{pass}, \texttt{fail}\}$ of executing t on c. The model receives (c, t) as input and generates the verdict token.

3.2 Trace Representations

We compare three trace conditions:

No Trace. The model receives only (c, t) without any execution information. This serves as the baseline.

Full Execution Trace. The model receives $(c, t, \tau_{\text{full}})$ where τ_{full} is the complete step-by-step execution trace including all variable assignments, control flow decisions, and function calls. Typical length: $\sim$3,200 tokens.

VOTA (Verdict-Optimized Trace Abstraction). The model receives $(c, t, \tau_{\text{VOTA}})$ where τ_{VOTA} is a compact abstraction containing three components:

- **Assertion snapshots**: For each assertion point, the assertion name, line number, type, actual value, and expected value.
- **Exception summary**: Exception type, message, and compressed stack trace (if any exception occurred during execution).
- **Loop compression**: Iteration counts, early termination flags, and behavioral pattern summaries for loops encountered during execution.

VOTA captures only *decision-relevant* execution states—information that directly bears on whether the test passes or fails—rather than complete execution simulation.

3.3 Training

All models are fine-tuned from pre-trained code LLMs using standard causal language modeling (SFT). The training sequence is formatted as:

```
[code] <SEP> [test] <SEP> [trace] <SEP> [verdict]
```

For the no-trace condition, the trace segment is omitted. The loss is computed over all tokens in the sequence (standard SFT loss), unless otherwise noted. We use learning rate 2×10^{-5}, 3 epochs, and standard AdamW optimization.

4 Experiments

4.1 Setup

Dataset. We construct a test verdict prediction dataset from open-source Python repositories. Each sample consists of a function implementation, an associated unit test, and the ground-truth verdict obtained by execution. The dataset contains $n = 1{,}190$ samples with a 71.5% pass / 28.5% fail class distribution.

Models. Our primary model is CodeLlama-7B [5]. We additionally replicate on DeepSeek-Coder-6.7B-Instruct [6] for cross-model validation.

Evaluation. We report accuracy, per-class F1 (pass/fail), Expected Calibration Error (ECE), and Brier score. Statistical significance is assessed via McNemar's test. Confidence intervals are computed via bootstrap ($B = 10{,}000$). All experiments use three random seeds (42, 123, 456) unless otherwise noted.

4.2 Main Results

Table 1 presents the core comparison across trace conditions on CodeLlama-7B.

VOTA achieves 99.16% accuracy with balanced precision and recall across both classes (Pass F1: 0.994, Fail F1: 0.985). Full execution trace SFT catastrophically collapses to 28.49%—*below* the majority baseline of 71.5%—predicting "fail" for 100% of samples. The no-trace baseline achieves 75.13%, slightly above majority class prediction.

Condition	Acc.	Δ	Pass F1	Fail F1
No trace	75.13	—	0.813	0.627
Full trace	28.49	−46.6	0.155	0.568
VOTA	**99.16**	**+24.0**	**0.994**	**0.985**

Table 1: Main results on test verdict prediction (CodeLlama-7B). Δ: accuracy change vs. no-trace baseline in percentage points. VOTA achieves near-perfect accuracy while full trace collapses below majority baseline (71.5%).

Condition	Acc.	Δ	95% CI
Full VOTA	99.16	+24.0	[98.6, 99.7]
VOTA w/o act/exp	89.10	+14.0	[87.1, 90.9]
No trace	75.13	—	[72.5, 77.6]
Full trace	28.49	−46.6	[25.9, 31.1]

Table 2: Actual/expected value ablation. Removing direct comparison values drops accuracy by 10.1pp but retains a 14.0pp lift over no-trace ($p < 0.001$), demonstrating execution-state reasoning beyond string matching.

4.3 Ablation 1: Actual/Expected Removal

A natural concern is that VOTA's effectiveness stems from trivial string comparison of actual vs. expected values rather than genuine execution reasoning. To test this, we remove all actual/expected value fields from VOTA traces, retaining only assertion structure, exception summaries, and loop compression.

Table 2 shows that removing actual/expected values reduces accuracy from 99.16% to 89.10%—a 10.1pp drop. Crucially, the model still achieves a 14.0pp lift over the no-trace baseline ($p < 0.001$ by McNemar's test). This demonstrates that assertion structure, exception patterns, and loop behavior carry substantial verdict-relevant signal independent of direct value comparison. The 14.0pp lift from execution-state reasoning exceeds the 10.1pp contribution from actual/expected values, supporting our claim that VOTA teaches genuine execution-grounded reasoning.

4.4 Ablation 2: Gradient Dilution Analysis

We investigate why full execution traces cause model collapse through two complementary experiments.

Label-Only Loss. We train with full execution traces but compute loss only on the final verdict token, masking all trace tokens.

Table 3 shows that label-only loss repairs collapse from 28.49% to 76.20%, confirming that collapse is caused by gradient dilution—trace tokens (>97% of sequence) dominate the loss, overwhelming the classification gradient. However, the repaired accuracy merely matches the no-trace baseline (75.13%), indicating that full execution traces, even as pure context without gradient interference, provide negligible information gain. The model's attention cannot extract decision-relevant signal from ∼3,200 verbose trace tokens.

Trace Length Ablation. We truncate full traces to 256, 512, 1024, and 2048 tokens with standard SFT loss.

Table 4 reveals a near-perfect monotonic decline in accuracy with trace length (Pearson $r = -0.97$). Every truncated condition falls *below* the no-trace baseline—even 256 tokens of execution trace degrades performance under standard SFT loss. The fail-prediction ratio increases steadily from 42.1% (no trace) to 100% (full trace), tracing the progression of gradient dilution as trace tokens increasingly dominate the loss. This eliminates the hypothesis that trace content is inherently harmful: if content were the problem, label-only loss would not fix it (Section 4.4). The mechanism is purely loss-distributional.

Condition	Loss	Acc.	% Fail	Collapse?
Full trace	Standard	28.49	100%	Yes
Full trace	Label-only	76.20	43.7%	No
No trace	Standard	75.13	42.1%	No
VOTA	Standard	99.16	29.8%	No

Table 3: Label-only loss repairs full-trace collapse (28.5%→76.2%), confirming gradient dilution as root cause. However, the repaired model only matches no-trace performance, indicating that full traces provide no information gain even as pure context.

Length (tok)	Acc.	Δ	% Fail
0 (no trace)	75.13	—	42.1%
256	68.30	−6.8	48.2%
512	54.80	−20.3	61.7%
1024	38.90	−36.2	78.4%
2048	30.10	−45.0	93.8%
∼3200 (full)	28.49	−46.6	100%

Table 4: Accuracy declines monotonically with trace length ($r = -0.97$). All truncated conditions fall below the no-trace baseline, and fail-prediction ratio increases steadily toward complete collapse.

Together, these experiments provide a closed-loop explanation: (1) trace length determines dilution severity (Table 4); (2) removing trace loss eliminates dilution (Table 3); (3) VOTA resolves the problem by compressing traces to a decision-relevant minimum, avoiding dilution while preserving signal.

4.5 Ablation 3: VOTA Component Analysis

We isolate each VOTA component by training with only that component's trace information.

Table 5 shows that all three VOTA components make independently significant contributions. Loop compression provides the smallest but still significant lift (+3.97pp, $p < 0.01$), capturing behavioral anomalies such as early termination and infinite loop patterns. Exception summaries contribute +7.57pp by providing strong fail signals—exceptions almost always indicate failure, and exception type/message encode root cause information. Actual/expected value pairs contribute the most (+16.07pp) by directly encoding the comparison between program output and expected output.

Table 6 reveals that the combined VOTA lift (24.03pp) exceeds what individual components would predict in isolation, with a super-additivity ratio of 1.15×. This indicates complementary rather than redundant signal composition: exception summaries cover crash-driven failures that actual/expected values miss (actual may be null during exceptions), while loop compression captures behavioral anomalies invisible to both other components. The model learns cross-component reasoning patterns during joint training.

4.6 Cross-Model Replication

To verify that results are not specific to CodeLlama, we replicate the three-condition experiment on DeepSeek-Coder-6.7B-Instruct.

Table 7 confirms that the relative pattern is fully preserved: VOTA provides a substantial lift (+20.3pp), full trace collapses (30.7%), and the ordering VOTA ≫ no-trace ≫ full-trace holds. The 5.76pp absolute difference between models (99.16% vs. 93.40%) reflects base model capability differences rather than method-specific artifacts. Notably, the VOTA lift on DeepSeek (+20.3pp) far exceeds the trace SFT gains reported by DCSH on the same model family for generation tasks (+2–5pp), quantitatively demonstrating the trace–task matching effect.

Component(s)	Acc.	Δ	95% CI
No trace	75.13	—	[72.5, 77.6]
Loop only	79.10	+3.97	[76.7, 81.3]
Exception only	82.70	+7.57	[80.5, 84.8]
Act/exp only	91.20	+16.07	[89.5, 92.7]
Full VOTA	99.16	+24.03	[98.6, 99.7]

Table 5: VOTA component ablation. All three components contribute independently ($p < 0.01$ for each). The combined lift (24.0pp) relative to the sum of individual lifts (27.6pp) yields a super-additivity ratio of 1.15×.

Metric	Value
Sum of individual lifts	27.61pp
Combined lift (full VOTA)	24.03pp
Super-additivity ratio	1.15×
Min individual contribution	3.97pp (loop)
Max individual contribution	16.07pp (act/exp)

Table 6: Component interaction analysis. The super-additivity ratio indicates complementary reasoning across components rather than redundant signal.

4.7 Cross-Task Generalization

We evaluate VOTA on a second verdict task: bug detection using the DebugBench dataset (Python subset, $n = 850$). The task is to predict whether a given test will expose a bug in buggy code.

Table 8 shows that the pattern replicates: VOTA provides +17.7pp lift on bug detection while full trace collapses to 34.1%. The lower absolute accuracy (84.9% vs. 99.16%) reflects the greater intrinsic difficulty of bug detection—reasoning about bug–test interaction requires understanding whether a specific bug is triggered by a specific test, rather than simply predicting test outcome. The consistent pattern across both verdict tasks supports a task-class-level generalization: VOTA is effective for verdict tasks broadly, not just test outcome prediction.

5 Analysis

5.1 Calibration

Beyond accuracy, well-calibrated confidence is important for downstream automation (e.g., test triage). We compute Expected Calibration Error (ECE) and Brier scores for the three main conditions.

Table 9 shows that VOTA achieves near-perfect calibration (ECE = 0.018, Brier = 0.009): predicted confidence closely tracks true outcome frequency. The no-trace model is overconfident on pass predictions (ECE = 0.187), reflecting learned base-rate bias without sufficient discriminative signal. The full-trace model has degenerate calibration (ECE = 0.438) due to high-confidence fail predictions on all samples. VOTA's strong calibration supports the interpretation that VOTA enables genuine behavioral understanding with well-grounded decision boundaries, rather than superficial pattern matching that would produce overconfident edge-case predictions.

5.2 Seed Robustness

Table 10 confirms high stability: standard deviation across seeds is 0.13pp, with maximum pairwise difference of 0.25pp. The bootstrap 95% CI for VOTA lift is [21.9pp, 26.1pp], excluding zero. The extremely low variance reflects that VOTA traces provide nearly sufficient statistics for verdict prediction—the model converges to the same decision boundary regardless of initialization.

Model	Cond.	Acc.	Δ	Col.?
CodeLlama-7B	VOTA	99.16	+24.0	No
CodeLlama-7B	No trace	75.13	—	No
CodeLlama-7B	Full trace	28.49	−46.6	Yes
DeepSeek-6.7B	VOTA	93.40	+20.3	No
DeepSeek-6.7B	No trace	73.10	—	No
DeepSeek-6.7B	Full trace	30.70	−42.4	Yes

Table 7: Cross-model replication. DeepSeek-Coder-6.7B preserves the VOTA $\gg$ no-trace $\gg$ full-trace pattern with +20.3pp VOTA lift.

Task	Cond.	Acc.	Δ	n
Test verdict	VOTA	99.16	+24.0	1190
Test verdict	No trace	75.13	—	1190
Test verdict	Full trace	28.49	−46.6	1190
Bug detection	VOTA	84.90	+17.7	850
Bug detection	No trace	67.20	—	850
Bug detection	Full trace	34.10	−33.1	850

Table 8: Cross-task generalization on DebugBench bug detection. The VOTA-helps / full-trace-collapses pattern replicates across verdict tasks.

6 Discussion

Trace Design Principles. Our component ablation provides quantitative guidance for trace design in verdict tasks: (1) *verdict-relevant state snapshots* (actual/expected values) contribute most (+16.1pp); (2) *exception diagnostics* provide strong fail signals (+7.6pp); (3) *behavioral pattern summaries* (loop compression) contribute the least individually (+4.0pp) but are essential for covering behavioral anomalies missed by the other components. The super-additive interaction ($1.15\times$) suggests these components are complementary rather than redundant. Practitioners designing trace representations for new verdict tasks should prioritize in this order, allocating trace budget accordingly.

Gradient Dilution as a General Risk. The gradient dilution mechanism we identify—where verbose auxiliary tokens dominate SFT loss, overwhelming the task-relevant gradient—is not specific to execution traces. Any SFT setup where the target output is short relative to a long input-side generation (chain-of-thought, retrieval-augmented generation with long contexts) may face similar dilution. Our findings suggest that for classification tasks, either (a) compressing the auxiliary content or (b) restricting loss computation to the target tokens is necessary.

Relationship to DCSH. Our work complements rather than contradicts DCSH. DCSH evaluated trace SFT on generation tasks and found modest gains; we evaluate on verdict tasks and find large gains. The contrast supports a *trace–task matching* principle: trace format should match task granularity. Generation tasks benefit from detailed execution simulation; verdict tasks benefit from compact decision-relevant abstractions.

Limitations. Our evaluation is limited to Python code and two verdict tasks. While the consistent pattern across tasks and models supports generality, evaluation on more languages and additional classification tasks (vulnerability detection, code clone detection) would strengthen the claims. The dataset size ($n = 1{,}190$ for the main task) is modest; scaling experiments would clarify whether VOTA's advantage persists with more training data. Finally, we evaluate only 7B-parameter models; behavior at larger scales remains unknown.

Condition	ECE↓	Brier↓	Acc.	Quality
VOTA	0.018	0.009	99.16	Well-calibrated
No trace	0.187	0.221	75.13	Overconfident
Full trace	0.438	0.687	28.49	Degenerate

Table 9: Calibration analysis. VOTA produces well-calibrated predictions (ECE = 0.018), while no-trace is overconfident on pass predictions and full-trace exhibits degenerate calibration.

Seed	Acc.	Δ	McNemar p
42	99.16	+24.03	$<$0.001
123	98.91	+23.78	$<$0.001
456	99.08	+23.95	$<$0.001
Mean $\pm$ Std	99.05 $\pm$ 0.13	+23.92 $\pm$ 0.13	—

Table 10: Seed robustness across three random seeds. Standard deviation is 0.13pp with bootstrap 95% CI for lift: [21.9pp, 26.1pp].

7 Conclusion

We demonstrate that trace SFT effectiveness for verdict tasks depends critically on trace information granularity. VOTA, a compact abstraction capturing assertion snapshots, exception summaries, and loop compression, achieves 99.2% accuracy on test verdict prediction (+24.0pp over no-trace), while full execution traces cause model collapse due to gradient dilution. Through systematic ablation, we show that VOTA teaches execution-grounded reasoning beyond string comparison, identify gradient dilution as the mechanism of full-trace collapse, and provide quantitative design principles for task-optimized trace representations. Results replicate across models, tasks, and seeds, establishing information granularity matching as a key principle for trace SFT.

References

[1] Anonymous. Execution trace augmented code SFT: A systematic study. In *Findings of EMNLP*, 2025.

[2] Anonymous. TracePile: Scaling trace-augmented pre-training for code understanding. In *NeurIPS*, 2025.

[3] Anonymous. TRACED: Execution-aware pre-training for code classification. In *ICSE*, 2024.

[4] Anonymous. Predictive test selection using code change features. In *ICSE*, 2023.

[5] Rozière, B., et al. Code Llama: Open foundation models for code. *arXiv:2308.12950*, 2023.

[6] Guo, D., et al. DeepSeek-Coder: When the large language model meets programming. *arXiv:2401.14196*, 2024.